

Notebook

February 8, 2026

1 RSA Blinding Attack

1.1 Intro

RSA (named after its creators Ronald Linn Rivest, Adi Shamir and Leonard Adleman) is an example of asymmetric cryptosystem that can be used for secure communications and signing data. While it is slowly fading into obscurity and giving way to other asymmetric cryptography algorithms, it is still being used in many production systems, so there is merit in learning how it works. The basic principle behind it is as follows.

1.2 Groups and Fields

A group is a set $\mathbb{G}$ with a binary operation $*$, $\forall a, b \in \mathbb{G} \Rightarrow a * b \in \mathbb{G}$ and:

- Associativity ($\forall a, b, c \in \mathbb{G}, (a * b) * c = a * (b * c)$)
- Identity element ($\exists e \in \mathbb{G} : \forall a \in \mathbb{G}, a * e = e * a = a$)
- Inverse element ($\forall a \in \mathbb{G}, \exists b \in \mathbb{G} : a * b = e = b * a$)

A group is called abelian if its group operation is commutative:

- $\forall a, b \in \mathbb{G}, a * b = b * a$

So if we can do some operation on a set, all the resulting values are also on this set, the order of evaluation of expressions doesn't matter, there is some neutral element that doesn't change other elements and there is always an inverse element that helps you create a neutral element, then you have a group. If you can also swap any two elements to which you apply the group operation and the result stays the same, then this is an abelian group.

If we are using additive notation, then we call the group additive. If we are using multiplicative notation -> multiplicative group.

A very simple example of an additive group is a group of integers modulo some number n . For example, if $n = 5$, then the group consists of $\{0, 1, 2, 3, 4\}$. Whenever there is an overflow or underflow, we quickly add or subtract $k * n, k \in \mathbb{N}$ from the result to put it back into this set. As you can see 0 is the neutral element, 1 is the inverse of 4 and 2 is the inverse of 3. Associativity is obvious, and since you can swap elements in an expression any way you like, this is an abelian group.

You can think of Fields as groups with two operations (it's not exactly true, but it helps). Let's say you have a set with two operations $(+, *)$, then if it is a field it should be an abelian group for both operators, but with one omission: the neutral element of the addition group can't have an inverse (you can't divide by zero). There is also a distribution law: $a * (b + c) = a * b + a * c$.

As an example, let's look at F_n , where $n = 5$:

1. The addition is the same as we previously saw
2. The multiplication table is the following:

	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

You can see that there is a 1 in every non-zero row, so every element has an inverse. If we delete the row and column containing the zero element, then we'll have a group. This is the multiplicative group modulo n or $\mathbb{Z}_n^*$ that contains elements $\{1, 2, 3, 4\}$. And here we can observe an interesting feature of groups: if the order of the group (the number of elements) is non-prime, then we can create subgroups. These are groups that use the same operation (for example, multiplication modulo 5), but they are subsets of elements of the original group. In the case of our group we can take the elements $\{1, 4\}$ and form a subgroup out of them since $4 \cdot 4 = 1 \pmod 5$. Subgroups aid with discrete logarithm. Because of this if we have to choose multiplicative groups modulo p for signing/encryption we take a large secure prime $p = 2 * q + 1$, where q is another prime. Then we only have two non-trivial subgroups with orders 2 and q .

1.3 RSA

RSA uses the multiplicative group modulo $N = pq$, where p and q are prime numbers. The order of the multiplicative group (effectively the number of elements in this group) can be calculated with Euler's totient function: $\varphi(N) = (p - 1)(q - 1)$. We just cross out the numbers less than N that are multiples of p or q .

Imagine $p = 13$ and $q = 17$. The element 26 can't be part of the multiplicative group for a very simple reason. If we multiply $26 \cdot 17$ we get a value that can be divided by $N = pq$, so $26 \cdot 17 = 0 \pmod{221}$. However, if we take out p elements $x : x = 0 \pmod q$ and q elements $y : y = 0 \pmod p$, then there will be no elements that could produce 0 during multiplication. So the number of multiplicative group members is $p \cdot q - p - q + 1 = (p - 1)(q - 1)$. We add 1, since we counted 0 twice ($0 = 0 \pmod p = 0 \pmod q$).

All the other numbers are in the multiplicative group, since they are coprime with N . As we know, if we exponentiate any member of a group to the order of the group, the result is group's identity: $a^{\varphi(N)} = 1, a \in \mathbb{Z}_N^*$. So we choose two numbers, e (public exponent) and d (private exponent) such that $ed = 1 \pmod{\varphi(N)}$. The pair of numbers

(e, N) becomes the public key and (d, N) becomes the private key. Calculating d from public information is considered a hard problem (unless certain mistakes were made). You could have to factor N to get p and q .

Given message $M, M < N$, public key (e, N) and private key (d, N) the encryption and decryption process is as follows:

Encryption $C = M^e \bmod N$

Decryption

$M = C^d \bmod N$

It's quite easy to check soundness:

$$C^d \bmod N = M^{ed} \bmod N = M^{ed \bmod \varphi(N)} \bmod N = M^1 \bmod N = M \bmod N$$

1.4 Preparation

Let's try to work with RSA for a bit. If you haven't yet, install Pycryptodome. On Linux and Windows this should work (You obviously have to install python 3 first and pip, but I hope you know how to do that):

```
[ ]: !python3 -m pip install --user pycryptodome
```

You'll have to restart jupyter kernel after installation (circular arrow near "Run"). If you encounter problems, you can follow this installation guide: [Pycryptodome installation](#).

1.5 Primitive RSA

So let's try to implement RSA. Let's use the public exponent $e = 65537$. This constant is usually chosen nowadays, because it wraps the modulus even if $M = 2$ and it has a nice binary representation $65537_{10} = 10000000000000001_2$ which allows for efficient exponentiation using the "Square and multiply" method.

First, generate the p and q . `getStrongPrime` function lets you choose the number of bits in your prime and checks that $\gcd(p-1, e) = 1$

```
[ ]: try:
      from Crypto.Util.number import getStrongPrime, inverse, bytes_to_long, long_to_bytes
    except ImportError:
      print ("Pycryptodome not installed")
```

```
[ ]: e=65537
      p=getStrongPrime(1024,e=e)
      q=getStrongPrime(1024,e=e)
```

```
[ ]: N=p*q
      phi=(p-1)*(q-1)
```

```
d=inverse(e,phi)
public_key=(e,N)
private_key=(d,N)
```

Ok, we've generated the keys, let's encrypt a message, decrypt the ciphertext and check if it is the same message

```
[ ]: M=bytes_to_long(b'Hello, RSA!')
C=pow(M,e,N)
print ('C:',hex(C))
M1=pow(C,d,N)
assert M1==M
print ('M1:',long_to_bytes(M1))
```

Creating a signature is inverse operation to encryption.

$$Sign(M) \equiv Dec(M), Check(S) \equiv Enc(S)$$

This way anyone with a public key can check the validity of the signature, but only the entity holding the private key can produce signatures. Congratulations. You know how to encrypt and create signatures with RSA. Now let's explore one of RSA's interesting properties.

1.6 RSA Blinding

RSA is homomorphic encryption under multiplication. Homomorphism is a structure-preserving map between two algebraic structures of the same type. (If you didn't get anything from that, that's ok. I also didn't the first time I heard that). What this means is that if you have two elements (x, y) and you put them through a homomorphic function, they will relate the same to each other in the new group/ring/field, etc. as they did in the original one:

$$\varphi(x \cdot y) = \varphi(x) \times \varphi(y)$$

This translates to

$$Enc(M_1 \cdot M_2) = Enc(M_1) \times Enc(M_2)$$

The same is true for decryptions:

$$Dec(C_1 \times C_2) = Dec(C_1) \cdot Dec(C_2)$$

Let's try this in python

```
[ ]: class BasicRSA:
    def __init__(self, e,p,q):
        self.e=e
        self.p=p
        self.q=q
        self.N=p*q
        self.d=inverse(e,(p-1)*(q-1))

    def encryptNumber(self, m):
```

```

        return pow(m,self.e,self.N)

    def decryptNumber(self, c):
        return pow(c,self.d,self.N)

brsa=BasicRSA(e,p,q) #we created these parameters earlier
m1=2
m2=3
m3=m1*m2
c1=brsa.encryptNumber(m1)
c2=brsa.encryptNumber(m2)
print ('c1:',c1)
print ('c2:',c2)
c3=(c1*c2)%brsa.N
print('c3:',c3)
m3_dec=brsa.decryptNumber(c3)
print ('m3: %d, m3_dec: %d'%(m3,m3_dec))
assert m3_dec==m3

```

1.7 Attacking the server

Now try to apply this knowledge to a vulnerable server. You can connect to it with nc cryptotraining.zone 1337 or by using python sockets.

```

[ ]: import socket
import re
class VulnServerClient:
    def __init__(self,show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1337))
        if show:
            print (self.recv_until().decode())
    def recv_until(self,symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def get_public_key(self,show=True):
        """Receive public key from the server"""
        self.s.sendall('public\n'.encode())
        response=self.recv_until().decode()
        if show:

```

```

        print (response)
    e=int(re.search(r'(?<=e: )\d+', response).group(0))
    N=int(re.search(r'(?<=N: )\d+', response).group(0))
    self.num_len=len(long_to_bytes(N))
    return (e,N)

def signBytes(self,m,show=True):
    """Get a signature for chosen byte message from the server"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪first')
        return
    if len(m)>num_len:
        print ("The message is too long")
        return
    if len(m)<num_len:
        m=bytes((num_len-len(m))*[0x0])+m
    hex_m=m.hex().encode()
    self.s.sendall(b'sign '+hex_m+b'\n')
    response=self.recv_until().decode()
    if show:
        print (response)
    if response.find('flag')!=-1:
        print('You tried to submit \'flag\'')
        return None
    signature_hex=re.search(r'(?<=Signature: )[0-9a-
f]+', response).group(0)
    signature_bytes=bytes.fromhex(signature_hex)
    return bytes_to_long(signature_bytes)

def signNumber(self,m,show=True):
    """Get a signature for chosen number from the server"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪first')
        return
    return self.signBytes(long_to_bytes(m,num_len),show)

def checkSignatureNumber(self,c,show=True):
    """Check if this number is a valid signature for 'flag'"""
    try:
        num_len=self.num_len

```

```

        except AttributeError:
            print ('You need to get the public key from the server_
↪first')
            return
        signature_bytes=long_to_bytes(c,num_len)
        self.checkSignatureBytes(signature_bytes,show)

    def checkSignatureBytes(self,c,show=True):
        """Check if these bytes are a valid signature for 'flag'"""
        try:
            num_len=self.num_len
        except AttributeError:
            print ('You need to get the public key from the server_
↪first')
            return
        if len(c)>num_len:
            print ("The message is too long")
            return

        hex_c=c.hex().encode()
        self.s.sendall(b'flag '+hex_c+b'\n',)
        response=self.recv_until(b'\n').decode()

        if show:
            print (response)

        if response.find('Wrong')!=-1:
            print('Wrong signature')
            x=self.recv_until()
            if show:
                print (x)
            return
        flag=re.search(r'CRYPTOTRAINING\{.*\}',response).group(0)
        print ('FLAG: ',flag)

    def __del__(self):
        self.s.close()

```

```

[ ]: vs=VulnServerClient()
      (e,N)=vs.get_public_key()

```

You can sign messages with signNumber and signBytes methods.

You can check signatures with checkSignatureNumber and checkSignatureBytes methods.

Your goal is to get the valid signature for message 'flag'

Remember that RSA is homomorphic and solve the task.

Good luck!

```
[ ]: vs.signBytes(b'flag')
```

```
[ ]:
```